

深度学习课程 Project 2 报告

任务 A: Instruction Fine-tuning 实践

小组成员：

2024214749 杨子木

2024214709 申婧

2024214739 康馨文

June 2026

摘要

本项目选择深度学习课程 Project 2 的任务 A: Instruction Fine-tuning 实践。我们参考 Stanford Alpaca 的 instruction tuning 思路，使用 Qwen2.5-0.5B 作为 base model，构造中英文混合 Alpaca-style instruction 数据集，并采用 LoRA 参数高效微调方法完成小规模监督微调实验。本实验先使用 1000 条中英文混合 instruction 数据完成 LoRA 微调，并进一步扩展到 3000 条样本进行数据规模对比实验。实验结果显示，在相同超参数设置下，3K 实验训练时间随数据量增加而增长，但峰值显存基本保持不变。实验记录了训练超参数、训练 loss、训练时间、显存占用，并对比了 base model 与 fine-tuned model 在中文问答、翻译、总结、代码解释、数学推理、逻辑判断和日常助手类任务上的输出差异。实验结果表明，小规模 LoRA instruction fine-tuning 能够改善模型的指令遵循能力和回答格式，但对复杂推理、代码理解和专业知识能力的提升仍然有限。

目录

1	项目选择与实验目标	4
2	模型与数据集	4
2.1	Base Model	4
2.2	数据集	4
3	Prompt Template 设计	5
4	微调方法：LoRA	6
5	实验环境与训练设置	7
5.1	实验环境	7
5.2	训练超参数	7
6	训练过程与结果	8
6.1	Mixed 1K 训练结果	9
6.2	Mixed 3K 训练结果	10
6.3	数据规模对比：Mixed 1K vs Mixed 3K	11
7	微调前后输出对比	12
7.1	翻译任务：指令遵循能力提升	12
7.2	总结任务：3K 模型对任务意图的理解略有提升	13
7.3	数学推理任务：3K 模型出现失败案例	13
7.4	逻辑判断任务：复杂推理能力提升有限	13
7.5	日常助手任务：3K 模型表现明显改善	14
8	能力变化分析	14
8.1	指令遵循能力明显提升	14
8.2	回答格式和助手风格有所改善	14
8.3	中英文翻译任务的指令识别能力提升，但生成控制仍不稳定	15
8.4	总结类任务有一定改善，但深层概括能力有限	15
8.5	复杂推理和代码理解能力提升有限	15
8.6	数据规模扩大并不必然带来全面提升	16
8.7	小结	16
9	失败案例分析	16
9.1	重复生成与截断	17
9.2	数学推理错误	17
9.3	逻辑判断失败	17

目录	3
10 总结与反思	17
11 小组成员与分工	19
12 GitHub 仓库说明	20

1 项目选择与实验目标

本组选择深度学习课程 Project 2 的**任务 A: Instruction Fine-tuning 实践**。本项目参考 Stanford Alpaca 的 instruction tuning 思路，完成一个小规模的大语言模型监督微调实验。实验目标是使用 Alpaca-style instruction-following 数据，对一个较小规模的开源 base model 进行 LoRA 参数高效微调，并比较微调前后模型在多种测试指令上的输出变化。本实验重点关注以下问题：

1. 如何将 instruction-following 数据整理为 Alpaca-style prompt 格式；
2. 如何使用 LoRA 方法对小型语言模型进行 instruction fine-tuning；
3. 微调前后模型在中文问答、翻译、总结、代码解释、数学推理、逻辑判断和日常助手类任务上的表现有何变化；
4. 小规模 instruction fine-tuning 的提升和局限分别是什么。

本项目最终提交 GitHub 仓库、训练代码、配置文件、训练日志、loss 曲线、推理对比结果和最终报告。

2 模型与数据集

2.1 Base Model

本实验选择 **Qwen2.5-0.5B** 作为 base model。选择该模型的主要原因如下：

1. 模型参数规模较小，适合在单张消费级或云平台 GPU 上进行实验；
2. 相比 7B 或更大模型，0.5B 模型训练成本更低，便于快速完成完整实验闭环；
3. 该模型属于 base model，而非 instruct model，更适合观察 instruction fine-tuning 前后的输出差异。

本实验没有对完整模型参数进行 full fine-tuning，而是采用 LoRA 方法只训练少量新增参数，从而降低显存需求和训练成本。

2.2 数据集

本实验使用中英文混合的 Alpaca-style instruction 数据。数据来源为 GPT-4-LLM Alpaca-GPT4 English + Chinese 数据。我们从英文数据和中文数据中分别抽取部分样本，构造中英文混合训练集和验证集。

数据划分如下表所示。

表 1: 数据集划分

实验设置	总训练样本数	英文训练样本数	中文训练样本数
Mixed 1K	1000	500	500
Mixed 3K	3000	1500	1500

其中, Mixed 1K 实验使用 50 条验证样本, 其中英文 25 条、中文 25 条; Mixed 3K 实验使用 100 条验证样本, 其中英文 50 条、中文 50 条。两个实验均使用相同的数据清洗规则, 即去除 instruction 或 output 为空的样本, 并过滤总长度超过 1800 字符的样本。

每条数据包含以下字段。

表 2: Alpaca-style 数据字段

字段	含义
instruction	用户希望模型完成的任务说明
input	完成任务所需的额外输入, 可以为空
output	期望模型生成的回答
lang	样本语言, 取值为 en 或 zh

数据样例如下。

Listing 1: Alpaca-style 数据样例

```
{
  "instruction": "请解释什么是过拟合。",
  "input": "",
  "output": "过拟合是指模型在训练数据上表现很好, 但在未见过的数据上表现较差的现象。",
  "lang": "zh"
}
```

3 Prompt Template 设计

本实验采用 Alpaca-style prompt template。根据 input 字段是否为空, 构造两种 prompt。当 input 为空时, prompt 格式如下:

Listing 2: 无 input 字段时的 Prompt Template

```
### Instruction:
{instruction}
```

```
### Response:
{output}
```

当 input 不为空时, prompt 格式如下:

Listing 3: 有 input 字段时的 Prompt Template

```
### Instruction:
{instruction}

### Input:
{input}

### Response:
{output}
```

在推理阶段,我们只输入 Instruction 和可选的 Input,并让模型从 ### Response: 后开始生成回答。这样的设计能够使模型学习 instruction-following 的基本交互格式。

4 微调方法: LoRA

本实验使用 LoRA 进行参数高效微调。LoRA 的核心思想是在原模型权重保持冻结的情况下,为部分线性层加入低秩可训练矩阵。训练时只更新这些低秩矩阵参数,而不是更新完整模型参数。

相比 full fine-tuning, LoRA 具有以下优点:

1. 可训练参数量少,显存占用较低;
2. 训练速度较快;
3. 便于保存和共享 adapter;
4. 可以在不修改原始 base model 权重的情况下完成模型适配。

本实验对 Qwen2.5-0.5B 的注意力层和前馈网络相关模块加入 LoRA, 包括:

Listing 4: LoRA target modules

```
q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
```

训练过程中模型参数统计如下:

表 3: LoRA 参数统计

项目	数值
总参数量	498,431,872
可训练参数量	4,399,104
可训练参数比例	0.8826%

可以看到，本实验只训练了不到 1% 的参数，体现了 LoRA 参数高效微调的特点。

5 实验环境与训练设置

5.1 实验环境

本实验在 AutoDL 云平台上完成，主要硬件和软件环境如下。

表 4: 实验环境

项目	配置
GPU	NVIDIA GeForce RTX 5090
PyTorch	2.8.0 + CUDA 12.8
Transformers	5.12.1
微调框架	Transformers + PEFT
训练方式	LoRA

5.2 训练超参数

本实验正式训练使用的主要超参数如下。

表 5: 训练超参数

超参数	数值
Base model	Qwen2.5-0.5B
Training samples	1000
Epochs	2
Max sequence length	512
Per-device batch size	2
Gradient accumulation steps	8
Effective batch size	16
Learning rate	2e-4
LoRA r	8
LoRA alpha	16
LoRA dropout	0.05

训练命令如下：

Listing 5: 训练命令

```
python train_lora.py 2>&1 | tee results/training_log_mixed_1k.txt
```

显存监控通过单独脚本完成：

Listing 6: 显存监控命令

```
python scripts/monitor_gpu.py \
  --interval 1 \
  --csv results/gpu_usage_mixed_1k.csv \
  --summary results/gpu_summary_mixed_1k.txt
```

6 训练过程与结果

本实验首先使用 1000 条中英文混合 Alpaca-style instruction 数据完成 LoRA 微调，记为 Mixed 1K 实验。在此基础上，为进一步观察数据规模对 instruction fine-tuning 的影响，我们将训练数据扩展到 3000 条，并在相同模型和相同超参数设置下进行了 Mixed 3K 实验。两个实验均使用 Qwen2.5-0.5B 作为 base model，并采用 LoRA 作为参数高效微调方法。

为了保证对比的公平性，Mixed 1K 和 Mixed 3K 实验保持以下设置不变：base model、LoRA target modules、LoRA rank、learning rate、max sequence length、per-device batch size、gradient accumulation steps 和训练 epoch 数。两组实验的主要区别仅在于训练样本规模不同。

6.1 Mixed 1K 训练结果

Mixed 1K 实验使用 1000 条训练样本，其中包含 500 条英文样本和 500 条中文样本。验证集包含 50 条样本，其中英文和中文样本各 25 条。该实验训练 2 个 epoch，主要训练结果如表 6 所示。

表 6: Mixed 1K 训练结果

指标	数值
训练样本数	1000
英文训练样本数	500
中文训练样本数	500
验证样本数	50
Epochs	2
Effective batch size	16
Learning rate	2e-4
Final train_loss	1.576
Trainer train_runtime	118.7 s
Wall time	129 s
Peak GPU memory	5650 MB
Peak GPU utilization	43%

Mixed 1K 实验的 loss 曲线如图 1 所示。从曲线可以看出，训练过程中 loss 整体处于下降趋势，说明模型能够从中英文混合 instruction 数据中学习到一定的 instruction-following 模式。由于训练数据规模较小，且每隔若干 step 记录一次 loss，曲线中存在一定波动，这是小规模微调实验中较常见的现象。

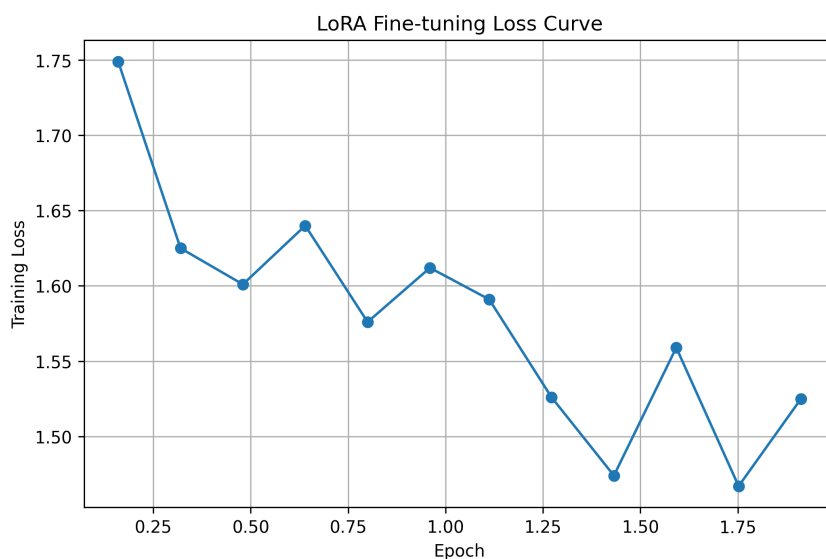


图 1: Mixed 1K LoRA Fine-tuning Loss Curve

6.2 Mixed 3K 训练结果

为了进一步观察数据规模扩大后的影响，我们在相同训练设置下进行了 Mixed 3K 实验。该实验使用 3000 条训练样本，其中英文样本 1500 条、中文样本 1500 条。验证集包含 100 条样本，其中英文和中文样本各 50 条。Mixed 3K 实验同样训练 2 个 epoch，主要结果如表 7 所示。

表 7: Mixed 3K 训练结果

指标	数值
训练样本数	3000
英文训练样本数	1500
中文训练样本数	1500
验证样本数	100
Epochs	2
Effective batch size	16
Learning rate	2e-4
Final train_loss	1.602
Trainer train_runtime	355.8 s
Wall time	367 s
Peak GPU memory	5650 MB
Peak GPU utilization	44%

Mixed 3K 实验的 loss 曲线如图 2 所示。训练过程中 loss 存在一定波动，但整体上从前期约 1.6–1.7 的区间下降到后期约 1.5–1.6 的区间，说明模型能够从 3000 条中英文混合 instruction 数据中学习到的指令响应模式。

曲线波动主要可能来自混合任务数据的差异：不同 batch 中样本语言、任务类型和难度并不完全一致，因此 step-level loss 不会严格单调下降。最终 Mixed 3K 实验的 train_loss 为 1.602，表明训练正常完成。

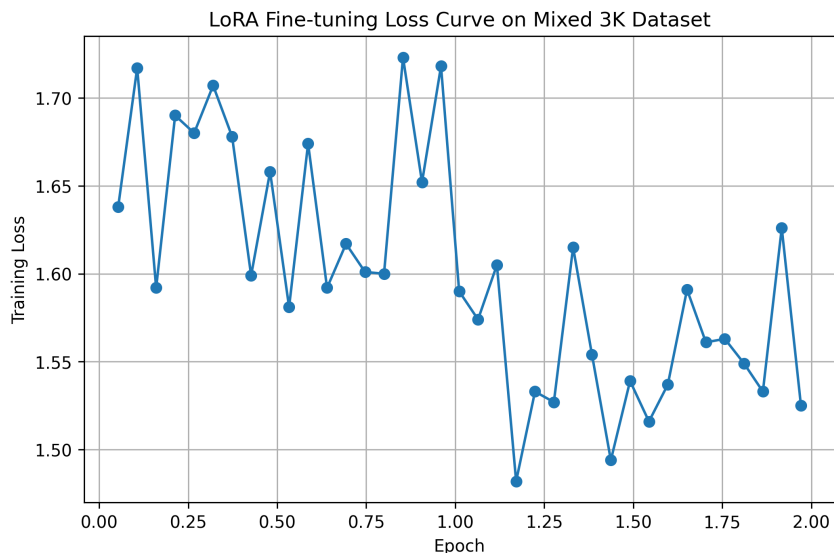


图 2: Mixed 3K LoRA Fine-tuning Loss Curve

6.3 数据规模对比: Mixed 1K vs Mixed 3K

为了更直观地比较数据规模对训练过程的影响, 表 8 汇总了 Mixed 1K 和 Mixed 3K 两组实验的关键指标。

表 8: Mixed 1K 与 Mixed 3K 数据规模实验对比

项目	Mixed 1K	Mixed 3K
训练样本数	1000	3000
英文训练样本数	500	1500
中文训练样本数	500	1500
验证样本数	50	100
Epochs	2	2
Effective batch size	16	16
Learning rate	2e-4	2e-4
Final train_loss	1.576	1.602
Trainer runtime	118.7 s	355.8 s
Wall time	129 s	367 s
Peak GPU memory	5650 MB	5650 MB
Peak GPU utilization	43%	44%

从训练时间来看, Mixed 3K 的 wall time 从 Mixed 1K 的 129 秒增加到 367 秒, 约为 1K 实验的 2.84 倍。这与训练样本数从 1000 增加到 3000 的变化基本一致, 说明在其他超参数保持不变的情况下, 数据集规模主要影响总训练时间。

从显存占用来看，两组实验的峰值显存均为 5650 MB。这说明在相同的 batch size、gradient accumulation steps 和 max sequence length 设置下，训练样本总数的增加不会明显提高单步训练的峰值显存占用。也就是说，数据集规模主要影响训练步数和总耗时，而显存需求更多由模型规模、batch size、序列长度和 LoRA 配置决定。

从 final train_loss 来看，Mixed 3K 的 train loss 为 1.602，略高于 Mixed 1K 的 1.576。这个结果并不能简单解释为 3K 模型效果更差。原因是 Mixed 3K 数据包含更多多样化的中英文 instruction-following 样本，任务分布更复杂，训练 loss 受到样本难度和任务多样性的影响。因此，对于 instruction fine-tuning 实验，不能只根据 final train_loss 判断模型能力，还需要结合后续的推理输出对比，观察模型在指令遵循、回答稳定性和任务泛化方面的变化。

总体来看，Mixed 3K 实验在显存占用基本不变的情况下，引入了更多 instruction 数据，使模型接触到更丰富的任务形式。后续输出对比将进一步分析数据规模扩大是否改善模型的实际回答质量。

7 微调前后输出对比

为了评估 instruction fine-tuning 的实际效果，本实验设计了 10 条测试指令，覆盖中文问答、翻译、总结、代码解释、数学推理、逻辑判断、英文问答和日常助手等任务类型。我们分别对原始 base model、Mixed 1K fine-tuned model 和 Mixed 3K fine-tuned model 进行推理测试。完整输出结果保存在 GitHub 仓库中的以下文件：

```
results/output_comparison.md
results/output_comparison_3k.md
```

其中，output_comparison.md 对应 Mixed 1K 实验结果，output_comparison_3k.md 对应 Mixed 3K 实验结果。报告中选取若干代表性样例进行展示和分析。

7.1 翻译任务：指令遵循能力提升

Instruction:

```
Please translate the following sentence into Chinese.
```

Input:

```
Instruction tuning helps language models better follow user requests.
```

在该任务中，base model 直接复制英文原文，没有执行翻译操作。这说明原始 base model 更像是在续写文本，而不是理解用户给出的任务指令。Mixed 1K fine-tuned model 能够正确识别翻译任务，并输出较为准确的中文翻译，说明 LoRA instruction fine-tuning 对翻译类指令的遵循能力有明显帮助。

Mixed 3K fine-tuned model 同样能够给出正确的中文翻译，但随后出现了较严重的重复生成问题，即模型在完成翻译后继续生成多段解释性文本，并多次重复相似内容。

这说明扩大数据规模后，模型的指令识别能力仍然存在，但生成控制能力并不稳定，可能会在简单任务中过度生成。

7.2 总结任务：3K 模型对任务意图的理解略有提升

在总结任务中，base model 和 Mixed 1K fine-tuned model 都主要将原文三句话重新编号输出，更多是在复述原文，而不是进行真正的概括。Mixed 3K fine-tuned model 在列出要点后，额外生成了一句总结性表达，概括了预训练、监督微调 and 偏好学习之间的关系。

这表明 Mixed 3K 模型相比 Mixed 1K 在摘要类任务上有一定改善：它不仅学习到了回答格式，也在一定程度上理解了“总结”这一任务意图。不过，该总结仍然较为保守，没有大幅压缩原文信息，说明小规模 LoRA 微调对深层语义概括能力的提升仍然有限。

7.3 数学推理任务：3K 模型出现失败案例

Instruction:

如果一个商品原价 200 元，现在打八折，请问现价是多少？请给出计算过程。

在该样例中，base model 和 Mixed 1K fine-tuned model 都能够得到正确答案 160 元。Mixed 1K fine-tuned model 还明确写出了 $200 \times 0.8 = 160$ 的计算过程，较好地满足了“给出计算过程”的要求。

但是，Mixed 3K fine-tuned model 在该任务中出现了严重错误。它先正确说明“打八折意味着商品价格是原价的 80%”，但随后又错误地使用了 $200 \times (1 - 0.80)$ ，最终得到 40 元。这说明模型学会了输出详细计算步骤的格式，但并没有真正稳定掌握数学含义。该样例说明，增加 instruction 数据规模并不一定能提升数学推理能力，甚至可能在某些样例上引入新的错误。

7.4 逻辑判断任务：复杂推理能力提升有限

Instruction:

请判断下面的推理是否正确，并说明原因：所有猫都会飞，小黑是猫，所以小黑会飞。

该样例是一个典型失败案例。Base model 直接判断该推理正确，未能识别前提本身不符合现实。Mixed 1K fine-tuned model 在形式上有所改善，能够给出“推理不正确”的判断，但分析理由并不严谨。Mixed 3K fine-tuned model 则再次判断该推理正确，并生成了大量重复且混乱的解释。

这说明小规模 instruction fine-tuning 主要改善的是回答格式和指令响应风格，并不能显著提升小模型的复杂逻辑推理能力。即使训练数据从 1K 增加到 3K，模型仍可能生成看似有条理但实际错误的推理文本。

7.5 日常助手任务：3K 模型表现明显改善

Instruction:

请写一封礼貌的邮件，请老师允许我延期提交作业。

在日常助手类任务中，Mixed 3K fine-tuned model 的表现明显优于 base model 和 Mixed 1K fine-tuned model。Base model 的邮件中存在“已经提交作业”和“无法按时完成作业”的矛盾，Mixed 1K fine-tuned model 也出现了“老师提交作业”这样的逻辑错误。相比之下，Mixed 3K fine-tuned model 能够较自然地表达“最近遇到困难，因此无法按时提交作业”，邮件结构完整，语气礼貌，逻辑更加清晰。

该样例说明，增加 instruction 数据规模对日常助手类任务有一定帮助，尤其是对邮件写作、礼貌表达和格式组织等任务，更多样本可以帮助模型生成更符合用户需求的回答。

8 能力变化分析

结合 Mixed 1K 和 Mixed 3K 两组实验的输出结果，可以发现 LoRA instruction fine-tuning 对模型能力的影响并不是简单的“全面提升”。微调后模型在指令遵循、回答格式和日常助手类任务上确实有所改善，但在复杂推理、代码理解、生成控制和事实准确性方面仍然存在明显不足。同时，将训练数据从 1K 扩展到 3K 后，模型在部分任务上表现更自然，但也出现了重复生成、截断和推理错误等新的问题。

8.1 指令遵循能力明显提升

微调前的 base model 更像一个普通语言模型，容易把 prompt 当作待续写文本处理。例如在翻译任务中，base model 直接复制原文，没有真正执行“翻译”这一指令。经过 LoRA instruction fine-tuning 后，Mixed 1K 和 Mixed 3K 模型都能够更稳定地识别翻译、问答、总结、邮件写作等任务类型，并从 `### Response:` 后生成回答。

这一现象说明 Alpaca-style instruction 数据能够帮助模型学习基本的 instruction-response 交互格式。即使只训练少量 LoRA 参数，模型也能够在输出形式上更加接近一个 instruction-following assistant。

8.2 回答格式和助手风格有所改善

微调后模型的回答通常更加完整，也更像面向用户的解释。比如在中文问答任务中，fine-tuned model 往往能够先给出概念定义，再进一步解释原因、影响或解决方法；在日常助手类任务中，模型能够生成较完整的邮件格式，包括称呼、正文、致谢和落款。

尤其在 Mixed 3K 实验中，日常助手类任务的表现明显优于 base model 和 Mixed 1K 模型。对于“请写一封礼貌的邮件，请老师允许我延期提交作业”这一任务，Mixed 3K 模型生成的邮件逻辑更清晰，语气更自然，能够正确表达“遇到困难导致无法按时

提交”的因果关系。这说明增加 instruction 数据规模有助于模型学习常见助手类任务的表达方式和格式规范。

8.3 中英文翻译任务的指令识别能力提升，但生成控制仍不稳定

在英译中和中译英任务中，base model 通常直接复制输入文本，没有执行翻译操作。Mixed 1K 模型能够给出较准确的翻译结果，说明 instruction fine-tuning 对翻译类任务的指令遵循能力提升明显。

但是，Mixed 3K 模型虽然也能先给出正确翻译，却在部分样例中继续生成大量解释性文本，并出现重复和截断。例如在英译中任务中，模型先正确翻译了句子，但随后反复解释这句话的含义；在中译英任务中，模型先给出正确英文翻译，却又输出与任务矛盾的说明。这说明 3K 数据带来了更多回答模式，但模型没有稳定学会何时停止生成。换言之，数据规模扩大改善了任务识别能力，却没有完全解决生成控制问题。

8.4 总结类任务有一定改善，但深层概括能力有限

在总结任务中，base model 和 Mixed 1K 模型主要将原文三句话编号输出，更多是在复述原文，而不是进行真正的概括。Mixed 3K 模型在列出原文要点后，额外生成了一句总结性表达，对预训练、监督微调和偏好学习之间的关系进行了概括。

这表明 Mixed 3K 模型相比 Mixed 1K 在摘要类任务上有一定改善，能够更好地理解“总结”这一任务意图。不过，该总结仍然较为保守，对原文信息的压缩程度有限，说明小规模 LoRA 微调对深层语义概括能力的提升仍然有限。

8.5 复杂推理和代码理解能力提升有限

本实验中最明显的局限出现在数学推理、逻辑判断和代码解释任务上。微调后模型虽然更倾向于输出分步骤解释或结构化分析，但内容不一定正确。

在数学推理任务中，Mixed 1K 模型能够正确计算打八折后的价格，并写出 $200 \times 0.8 = 160$ 的过程；但 Mixed 3K 模型反而将“打八折”错误理解为“减去 80%”，使用 $200 \times (1 - 0.80)$ 得到 40 元的错误结果。这说明模型可能学会了“写出详细计算过程”的形式，但并没有真正稳定掌握数学含义。

在逻辑判断任务中，base model 和 Mixed 3K 模型都错误地判断“所有猫都会飞，小黑是猫，所以小黑会飞”这一推理正确，并生成了看似有条理但实际混乱的解释。Mixed 1K 模型虽然判断为“不正确”，但分析理由也不够严谨。这说明 LoRA instruction fine-tuning 对复杂逻辑推理能力的提升非常有限。

在代码解释任务中，fine-tuned model 的回答通常更详细，但仍会出现技术细节不准确的问题。例如模型将 Python 的 `range()` 描述为“生成器函数”或“生成器对象”，这在严格意义上并不准确。由此可见，小规模 instruction tuning 更多改善的是回答形式，而不是显著提升代码理解的准确性。

8.6 数据规模扩大并不必然带来全面提升

从 Mixed 1K 到 Mixed 3K，训练样本数从 1000 增加到 3000，模型接触到了更多中英文 instruction-following 样本。实验结果显示，3K 模型在日常助手和总结任务上确实表现更好，回答更自然，格式也更完整。

然而，3K 模型也出现了一些 1K 模型中不明显的问题，例如翻译任务中的重复生成、数学任务中的错误推理、逻辑任务中的长篇错误分析。这说明增加数据规模并不一定带来所有能力的稳定提升。对于 Qwen2.5-0.5B 这样的小模型，更多 instruction 数据可以改善回答风格和常见任务格式，但复杂推理、事实准确性和生成控制仍然受到 base model 能力、数据质量和训练设置的限制。

8.7 小结

总体来看，LoRA instruction fine-tuning 对本实验模型的主要提升体现在以下方面：

1. 模型更容易识别用户指令，而不是简单续写 prompt；
2. 回答格式更加稳定，更接近 instruction-following assistant；
3. 中文问答、翻译、总结和日常助手类任务的表现有所改善；
4. 增加到 3K 数据后，日常助手类任务和部分总结任务表现更自然。

但模型仍然存在以下不足：

1. 数学推理和逻辑判断能力不稳定；
2. 代码解释中仍有技术细节错误；
3. 3K 模型在部分任务中出现重复生成和截断；
4. 微调主要改善回答风格，并不能显著突破 base model 本身的能力上限。

因此，本实验表明，小规模 LoRA instruction fine-tuning 可以有效改善模型的指令遵循能力和回答格式，但其效果具有明显边界。对于复杂推理、专业知识和生成稳定性问题，仅增加少量 instruction 数据并不足够，还需要更高质量的数据、更合适的训练策略以及更系统的评估方法。

9 失败案例分析

本实验中特别关注失败案例，而不是只展示效果较好的样例。失败案例主要包括以下几类：

9.1 重复生成与截断

在 Mixed 3K 的翻译任务中，模型能够先给出正确翻译，但随后反复生成解释性内容，甚至出现截断。这说明模型虽然学会了执行翻译指令，但在何时停止生成方面仍然不稳定。

9.2 数学推理错误

在商品打折计算任务中，Mixed 3K 模型将“打八折”错误地计算为 $200 \times (1 - 0.80)$ ，最终得到 40 元的错误答案。这说明模型可能学会了输出分步计算格式，但并不真正理解折扣含义。

9.3 逻辑判断失败

在“所有猫都会飞”的逻辑判断任务中，Mixed 3K 模型错误地判断推理正确，并生成大量重复解释。这说明 instruction fine-tuning 无法显著弥补小模型在逻辑推理能力上的不足。

10 总结与反思

本项目完成了一个小规模 Alpaca-style instruction fine-tuning 实验。我们使用 Qwen2.5-0.5B 作为 base model，采用 LoRA 参数高效微调方法，分别在 Mixed 1K 和 Mixed 3K 两组中英文混合 instruction 数据上进行了监督微调实验。实验过程中，我们完成了数据构造、prompt template 设计、LoRA 训练、训练日志记录、显存监控、loss 曲线绘制以及 base model 与 fine-tuned model 的输出对比分析。

从训练过程来看，LoRA 微调能够以较低的资源成本完成。Mixed 1K 实验的训练 wall time 为 129 秒，Mixed 3K 实验的训练 wall time 为 367 秒；当训练样本数从 1000 增加到 3000 时，训练时间相应增加，但两组实验的峰值显存均为 5650 MB。这说明在相同 batch size、max sequence length 和 LoRA 配置下，数据规模主要影响训练步数和总训练时间，而不会明显增加单步训练的显存占用。

从模型输出效果来看，instruction fine-tuning 对模型的指令遵循能力和回答格式有明显帮助。相比 base model，fine-tuned model 更容易理解翻译、问答、总结和邮件写作等任务指令，也更倾向于生成符合 Instruction-Response 格式的回答。在翻译任务和日常助手类任务中，微调后的模型表现出更明显的提升，说明 Alpaca-style instruction 数据能够有效改善模型的助手式回答风格。

不过，实验也表明，增加训练数据规模并不必然带来所有任务上的全面提升。Mixed 3K 模型在日常助手类任务上表现更自然，邮件内容更加连贯，但在部分翻译任务中出现了重复生成和截断问题；在数学推理任务中，Mixed 3K 模型甚至将“打八折”错误计算为 $200 \times (1 - 0.80)$ ，得到错误答案；在逻辑判断任务中，模型仍然无法稳定识别推理中的问题，并生成了看似有条理但实际错误的解释。这说明小规模 instruction fine-tuning

主要改善的是模型“如何回答”的形式，而不能显著提升模型本身缺乏的复杂推理和精确理解能力。

本实验的主要结论可以总结为以下几点：

1. LoRA instruction fine-tuning 能够在较低显存占用下完成小模型的指令微调；
2. 微调后模型在指令遵循、回答格式和日常助手风格方面有明显改善；
3. Mixed 3K 相比 Mixed 1K 在部分任务上表现更自然，但并非所有任务都更好；
4. 数学推理、逻辑判断和代码解释等任务仍然存在明显不足；
5. 训练 loss 不能作为判断模型能力的唯一指标，必须结合具体输出样例进行分析。

本项目也存在一些局限。首先，训练数据规模仍然较小，即使 Mixed 3K 实验也只有 3000 条训练样本，远小于真实 instruction tuning 中常用的数据规模。其次，本实验主要依赖人工设计的 10 条测试指令进行输出对比，没有使用系统化 benchmark 或自动评估指标。第三，训练数据只进行了简单清洗，主要过滤空字段和过长样本，没有进一步检查数据质量、任务分布和回答准确性。最后，本实验使用的是 0.5B 规模的小模型，其基础知识和推理能力本身有限，因此 LoRA 微调难以显著突破 base model 的能力上限。

如果后续继续改进，可以从以下几个方向展开：

1. 扩大训练数据规模，例如尝试 5000 条或更多 instruction 数据；
2. 对训练数据进行更细致的质量筛选，减少重复、低质量或风格异常的样本；
3. 在推理阶段调整生成参数，例如限制最大生成长度、加入重复惩罚或合理设置停止条件，以减少重复生成和截断问题；
4. 尝试不同的 LoRA 超参数，例如 LoRA rank、learning rate 和训练 epoch 数；
5. 引入更系统的评估方法，例如构造固定测试集，或者使用 evaluation harness 对模型进行自动化评估；
6. 尝试更强的 base model，对比模型规模对 instruction fine-tuning 效果的影响。

总体而言，本实验完成了任务 A 要求的 instruction fine-tuning 基本流程。通过 Mixed 1K 和 Mixed 3K 两组实验，我们不仅观察到了 LoRA 微调对模型指令遵循能力的提升，也分析了小规模微调在复杂推理、代码理解和生成稳定性方面的不足。这说明 instruction fine-tuning 是改善语言模型交互风格的重要方法，但模型能力的真正提升仍然依赖于 base model 能力、数据质量、训练策略和评估方法等多方面因素。

11 小组成员与分工

表 9: 小组成员与分工

成员	分工
杨子木	环境配置、模型下载、LoRA 训练、训练日志整理
申婧	数据集整理、prompt template 设计、测试指令设计
康馨文	推理结果分析、失败案例分析、报告撰写

12 GitHub 仓库说明

GitHub 仓库中包含以下内容：

```
README.md
requirements.txt
train_lora.py
inference_compare.py
configs/
    train_config.yaml
data/
    train.json
    eval.json
    dataset_stats.json
    train_3k.json
    eval_3k.json
    dataset_stats_3k.json
examples/
    test_prompts.json
scripts/
    prepare_mixed_alpaca.py
    prepare_mixed_alpaca_3k.py
    plot_loss_any.py
results/
    training_log_mixed_1k.txt
    train_time_mixed_1k.txt
    gpu_summary_mixed_1k.txt
    gpu_usage_mixed_1k.csv
    loss_curve_mixed_1k.png
    output_comparison.md
    output_comparison.json
    run_summary_mixed_1k.md
    training_log_mixed_3k.txt
    train_time_mixed_3k.txt
    gpu_summary_mixed_3k.txt
    gpu_usage_mixed_3k.csv
    loss_curve_mixed_3k.png
    output_comparison_3k.md
    output_comparison_3k.json
    run_summary_mixed_3k.md
report/
    project_report.pdf
```

完整 base model 权重未上传到 GitHub。仓库主要提交训练代码、配置文件、处理后的数据、训练日志、loss 曲线和输出对比结果。

参考文献

- [1] Stanford CRFM. *Alpaca: A Strong, Replicable Instruction-Following Model*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>
- [2] Hu, E. J., Shen, Y., Wallis, P., et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021.
- [3] Hugging Face. *Transformers Documentation*. <https://huggingface.co/docs/transformers>
- [4] Hugging Face. *PEFT Documentation*. <https://huggingface.co/docs/peft>
- [5] Qwen Team. *Qwen2.5 Model Documentation*. <https://qwenlm.github.io/>
- [6] Instruction Tuning with GPT-4. *GPT-4-LLM Dataset*. <https://github.com/Instruction-Tuning-with-GPT-4/GPT-4-LLM>